AI & AD Practical Journal

Code and Output for all practicals

Practical 1(A)

Aim: Design a simple rational agent by defining PEAS for the vacuum cleaner environment and autonomous taxi, classify environments, and implement a simple table-driven agent in Python.

Code

```
table = {
    ('A', 'Dirty'): 'Suck',
    ('A', 'Clean'): 'Right',
    ('B', 'Dirty'): 'Suck',
    ('B', 'Clean'): 'Left',
}

def table_driven_agent(location, status):
    percept = (location, status)
    if percept in table:
        return table[percept]
    else:
        return "No action"

print("Percept: (A, Dirty)")
print("Action:", table_driven_agent('A', 'Dirty'))

print("Percept: (A, Clean)")
print("Action:", table_driven_agent('A', 'Clean'))

print("Percept: (B, Clean)")
print("Action:", table_driven_agent('B', 'Clean'))

print("Percept: (B, Dirty)")
print("Action:", table_driven_agent('B', 'Dirty'))
```

Output

```
Percept: (A, Dirty)
Action: Suck
Percept: (A, Clean)
Action: Right
Percept: (B, Clean)
Action: Left
Percept: (B, Dirty)
Action: Suck
```

Practical 2(A)

Aim: Given an initial configuration of the puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search.

Code

```
from collections import deque

# Initial State
start = (
    (1, 2, 3),
    (4, 0, 6),
    (7, 5, 8)
)

# Goal State
goal = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))
    return neighbors

def bfs(start, goal):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        if state in visited:
            continue

        visited.add(state)

        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))

    return None

solution = bfs(start, goal)

if solution:
    print("Shortest path found")
    print("Number of moves:", len(solution) - 1)
    for step, state in enumerate(solution):
```

```
        print("\nStep", step)
        for row in state:
            print(row)
else:
    print("No solution found")
```

Output

```
Shortest path found
Number of moves: 2

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Practical 2(B)

Aim: Given two water jugs of 4 and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS).

Code

```
def dfs(jug1_capacity, jug2_capacity, target):
    stack = [((0, 0), [])]          # ((jug1, jug2), path)
    visited = set()

    while stack:
        (jug1, jug2), path = stack.pop()

        if (jug1, jug2) in visited:
            continue

        visited.add((jug1, jug2))
        path = path + [(jug1, jug2)]

        # Goal test
        if jug1 == target or jug2 == target:
            print("Solution found:\n")
            for step in path:
                print(step)
            return

        # Generate all possible next states
        next_states = []

        # Fill jug1
        next_states.append((jug1_capacity, jug2))
        # Fill jug2
        next_states.append((jug1, jug2_capacity))
        # Empty jug1
        next_states.append((0, jug2))
        # Empty jug2
        next_states.append((jug1, 0))

        # Pour jug1 -> jug2
        transfer = min(jug1, jug2_capacity - jug2)
        next_states.append((jug1 - transfer, jug2 + transfer))
        # Pour jug2 -> jug1
```

```
        transfer = min(jug2, jug1_capacity - jug1)
        next_states.append((jug1 + transfer, jug2 - transfer))

        # Push unvisited states
        for state in next_states:
            if state not in visited:
                stack.append((state, path))

    print("No solution exists.")

# Main program
jug1 = 4
jug2 = 3
target = 2

dfs(jug1, jug2, target)
```

Output

```
Solution found:

(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

Practical 3(A)

Aim: Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan distance.

Code

```
from queue import PriorityQueue

# Goal State
goal = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]

# Find the position of a value in the goal state
def find_position(value):
    for i in range(3):
        for j in range(3):
            if goal[i][j] == value:
                return i, j

# Manhattan Distance Heuristic
def manhattan(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0:
                x, y = find_position(state[i][j])
                distance += abs(i - x) + abs(j - y)
    return distance

# Convert list to tuple (for hashing)
def to_tuple(state):
    return tuple(tuple(row) for row in state)

# Find blank tile
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
```

```
                return i, j

# Generate neighboring states
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]   # up, down, left, right

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)
    return neighbors

# Print puzzle state
def print_state(state):
    for row in state:
        print(row)
    print()

# Greedy best-first search
def greedy_best_first(start):
    pq = PriorityQueue()
    pq.put((manhattan(start), start))

    visited = set()

    while not pq.empty():
        h, current = pq.get()

        if to_tuple(current) in visited:
            continue

        visited.add(to_tuple(current))

        print("Heuristic =", h)
        print_state(current)

        if current == goal:
            print("Goal state reached!")
            return

        for neighbor in get_neighbors(current):
            if to_tuple(neighbor) not in visited:
                pq.put((manhattan(neighbor), neighbor))

    print("No solution found.")

# Initial State
start = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]

greedy_best_first(start)
```

Output

```
Heuristic = 2
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Heuristic = 1
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
```

```
Heuristic = 0
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

Goal state reached!
```

Practical 3(B)

Aim: Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest path using A* search with the Manhattan distance heuristic.

Code

```
from queue import PriorityQueue

# Initial State
initial = (
    (1, 2, 3),
    (4, 0, 6),
    (7, 5, 8)
)

# Goal State
goal = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

# Store goal positions
goal_pos = {}
for i in range(3):
    for j in range(3):
        goal_pos[goal[i][j]] = (i, j)

# Manhattan Distance Heuristic
def manhattan(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                x, y = goal_pos[value]
                distance += abs(i - x) + abs(j - y)
    return distance

# Find blank tile (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate neighboring states
def neighbors(state):
    x, y = find_blank(state)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    result = []

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [list(row) for row in state]
            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]
            result.append(tuple(tuple(row) for row in temp))

    return result
```

```
# A* search algorithm
def a_star(start, goal):
    pq = PriorityQueue()

    # (f = g + h, g, state, path)
    pq.put((manhattan(start), 0, start, [start]))

    visited = {}

    while not pq.empty():
        f, g, state, path = pq.get()

        if state == goal:
            return path

        if state in visited and visited[state] <= g:
            continue

        visited[state] = g

        for next_state in neighbors(state):
            new_g = g + 1
            new_h = manhattan(next_state)
            new_f = new_g + new_h

            pq.put((new_f, new_g, next_state, path + [next_state]))

    return None

# Execute A* search
solution = a_star(initial, goal)

print("Solution path:\n")

for step, state in enumerate(solution):
    print("Step", step)
    for row in state:
        print(row)
    print()

print("Total moves:", len(solution) - 1)
```

Output

```
Solution path:

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Total moves: 2
```

Practical 3(C)

Aim: Given a weighted graph representing cities and distances between them, write a Python program to find the shortest path from Arad to Bucharest using A* search with a heuristic function.

Code

```
from queue import PriorityQueue

# Romania Road Map (Weighted Graph)
graph = {
    'Arad': [('Zerind', 75), ('Timisoara', 118), ('Sibiu', 140)],
    'Zerind': [('Arad', 75), ('Oradea', 71)],
    'Oradea': [('Zerind', 71), ('Sibiu', 151)],
    'Timisoara': [('Arad', 118), ('Lugoj', 111)],
    'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
    'Craiova': [('Drobeta', 120), ('Pitesti', 138), ('Rimnicu Vilcea', 146)],
    'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
    'Rimnicu Vilcea': [('Sibiu', 80), ('Pitesti', 97), ('Craiova', 146)],
    'Pitesti': [('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest': [('Fagaras', 211), ('Pitesti', 101)]
}

# Straight-line Distance Heuristic
heuristic = {
    'Arad': 366,
    'Zerind': 374,
    'Oradea': 380,
    'Timisoara': 329,
    'Lugoj': 244,
    'Mehadia': 241,
    'Drobeta': 242,
    'Craiova': 160,
    'Sibiu': 253,
    'Fagaras': 176,
    'Rimnicu Vilcea': 193,
    'Pitesti': 100,
    'Bucharest': 0
}

def a_star(start, goal):
    pq = PriorityQueue()

    # (f = g + h, g, node, path)
    pq.put((heuristic[start], 0, start, [start]))

    visited = {}

    while not pq.empty():
        f, g, node, path = pq.get()

        if node == goal:
            return path, g

        if node in visited and visited[node] <= g:
            continue

        visited[node] = g

        for neighbor, cost in graph[node]:
            new_g = g + cost
            new_f = new_g + heuristic[neighbor]
            pq.put((new_f, new_g, neighbor, path + [neighbor]))

    return None, float("inf")
start = "Arad"
```

```
goal = "Bucharest"

path, cost = a_star(start, goal)

print("Shortest Path:")
print(" -> ".join(path))
print("Total Cost:", cost)
```

Output

```
Shortest Path:
Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Cost: 418
```

Practical 4

Aim: Bayes' Rule application (probability of disease given a positive test).

Code

```
# Bayes' Rule Application

# Given Probabilities
P_D = 0.01            # Probability of having the disease
P_not_D = 0.99        # Probability of not having the disease

P_Pos_given_D = 0.99      # Test is positive if disease is present
P_Pos_given_not_D = 0.05  # False positive rate

# Bayes' Theorem
# P(Disease | Positive) =
# (P(Positive | Disease) * P(Disease)) /
# (P(Positive | Disease) * P(Disease) +
#  P(Positive | No Disease) * P(No Disease))

numerator = P_Pos_given_D * P_D
denominator = numerator + (P_Pos_given_not_D * P_not_D)

P_D_given_Pos = numerator / denominator

# Display Result
print("Probability that the person actually has the disease")
print("given a positive test result =", round(P_D_given_Pos, 4))

print("Percentage:", round(P_D_given_Pos * 100, 2), "%")
```

Output

```
Probability that the person actually has the disease
given a positive test result = 0.1667
Percentage: 16.67 %
```

Practical 5

Aim: Naive Bayes classification using the weather dataset.

Code

```
# Naive Bayes Classification using Weather Dataset

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

# ---------------------------------------
# Step 1: Create Weather Dataset
# ---------------------------------------
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain',
                'Rain', 'Rain', 'Overcast', 'Sunny',
                'Sunny', 'Rain', 'Sunny', 'Overcast',
                'Overcast', 'Rain'],

    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
                    'Cool', 'Cool', 'Mild', 'Cold', 'Mild',
                    'Mild', 'Mild', 'Hot', 'Mild'],

    'Humidity': ['High', 'High', 'High', 'High', 'Normal',
                 'Normal', 'Normal', 'High', 'Normal',
                 'Normal', 'Normal', 'High', 'Normal',
                 'High'],

    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Strong'],

    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes',
             'No', 'Yes', 'No', 'Yes', 'Yes',
             'Yes', 'Yes', 'Yes', 'No']
}

# Convert dictionary to DataFrame
df = pd.DataFrame(data)

print("Original Dataset:\n")
print(df)

# ---------------------------------------
# Step 2: Encode Categorical Data
# ---------------------------------------
label_encoders = {}

for column in df.columns:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le

print("\nEncoded Dataset:\n")
print(df)

# ---------------------------------------
# Step 3: Separate Features and Target
# ---------------------------------------
X = df[['Outlook', 'Temperature', 'Humidity', 'Wind']]
y = df['Play']

# ---------------------------------------
# Step 4: Train Naive Bayes Model
# ---------------------------------------
model = GaussianNB()
model.fit(X, y)
# ---------------------------------------
```

```
# Step 5: New Weather Sample
# ----------------------------------------
new_data = pd.DataFrame({
    'Outlook': ['Sunny'],
    'Temperature': ['Cool'],
    'Humidity': ['High'],
    'Wind': ['Strong']
})

# Encode the new sample
for column in new_data.columns:
    new_data[column] = label_encoders[column].transform(new_data[column])

# ----------------------------------------
# Step 6: Predict
# ----------------------------------------
prediction = model.predict(new_data)

# Convert prediction back to Yes/No
result = label_encoders['Play'].inverse_transform(prediction)

print("\nPrediction for New Weather Condition")
print("-------------------------------------")
print("Outlook: Sunny")
print("Temperature: Cool")
print("Humidity: High")
print("Wind: Strong")
print("\nPlay =", result[0])
```

Output

```
Original Dataset:

     Outlook Temperature Humidity    Wind Play
0      Sunny         Hot     High    Weak   No
1      Sunny         Hot     High  Strong   No
2   Overcast         Hot     High    Weak  Yes
3       Rain        Mild     High    Weak  Yes
4       Rain        Cool   Normal    Weak  Yes
5       Rain        Cool   Normal  Strong   No
6   Overcast        Cool   Normal  Strong  Yes
7      Sunny        Mild     High    Weak   No
8      Sunny        Cold   Normal    Weak  Yes
9       Rain        Mild   Normal    Weak  Yes
10     Sunny        Mild   Normal  Strong  Yes
11  Overcast        Mild     High  Strong  Yes
12  Overcast         Hot   Normal    Weak  Yes
13      Rain        Mild     High  Strong   No

Encoded Dataset:

    Outlook  Temperature  Humidity  Wind  Play
0         2            2         0     1     0
1         2            2         0     0     0
2         0            2         0     1     1
3         1            3         0     1     1
4         1            1         1     1     1
5         1            1         1     0     0
6         0            1         1     0     1
7         2            3         0     1     0
8         2            0         1     1     1
9         1            3         1     1     1
10        2            3         1     0     1
11        0            3         0     0     1
12        0            2         1     1     1
13        1            3         0     0     0

Prediction for New Weather Condition
-------------------------------------
Outlook: Sunny
Temperature: Cool
```

```
Humidity: High
Wind: Strong

Play = No
```

Practical 6(A)

Aim: Bayesian network (Burglary-Alarm): probability of Burglary given JohnCalls = 1 and MaryCalls = 1.

Code

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])

cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]]
)

cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]]
)

cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.06, 0.71, 0.05],
        [0.001, 0.94, 0.29, 0.95]
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2]
)

cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10],
        [0.05, 0.90]
    ],
    evidence=["Alarm"],
    evidence_card=[2]
)

cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30],
        [0.01, 0.70]
    ],
    evidence=["Alarm"],
    evidence_card=[2]
)

model.add_cpds(
    cpd_burglary,
```

```
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)

print("Model is Valid:", model.check_model())

inference = VariableElimination(model)

result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": 1,
        "MaryCalls": 1
    }
)

print("\nNetwork Edges:")
print(model.edges())

print("\nProbability of Burglary given John and Mary called:")
print(result)
```

Output

```
Model is Valid: True

Network Edges:
[('Burglary', 'Alarm'), ('Earthquake', 'Alarm'), ('Alarm', 'JohnCalls'), ('Alarm', 'MaryCalls')]

Probability of Burglary given John and Mary called:
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
| Burglary(0) |          0.9262 |
+-------------+-----------------+
| Burglary(1) |          0.0738 |
+-------------+-----------------+
```

Output computed by exact enumeration of the same network, shown in pgmpy's table format (pgmpy could not be installed in this environment).

Practical 6(B)

Aim: Bayesian network (Burglary-Alarm): posterior probability P(Burglary | Alarm = True).

Code

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])

cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]]
)

cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]]
```

```
)

cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.06, 0.71, 0.05],
        [0.001, 0.94, 0.29, 0.95]
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2]
)

cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10],
        [0.05, 0.90]
    ],
    evidence=["Alarm"],
    evidence_card=[2]
)

cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30],
        [0.01, 0.70]
    ],
    evidence=["Alarm"],
    evidence_card=[2]
)

model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)

print("Model is Valid:", model.check_model())

inference = VariableElimination(model)

result = inference.query(
    variables=["Burglary"],
    evidence={
        "Alarm": 1
    }
)

print("\nNetwork Edges:")
print(model.edges())

print("\nPosterior Probability P(Burglary | Alarm = True):")
print(result)
```

Output

```
Model is Valid: True

Network Edges:
[('Burglary', 'Alarm'), ('Earthquake', 'Alarm'), ('Alarm', 'JohnCalls'), ('Alarm', 'MaryCalls')]

Posterior Probability P(Burglary | Alarm = True):
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
```

```
| Burglary(0) |          0.9080 |
+-------------+-----------------+
| Burglary(1) |          0.0920 |
+-------------+-----------------+
```

Output computed by exact enumeration of the same network, shown in pgmpy's table format (pgmpy could not be installed in this environment).

Corrections made while transcribing

• **1(A):** The notebook printed the labels (B, Clean) and (B, Dirty) but called the agent with the opposite percepts; the calls now match their labels.

• **2(A):** Start state had a repeated 4 in the bottom row; changed to 7. Fixed `neighbors = 0` to `[]`, `starts` to `state`, and added the missing `find_blank` helper.

• **2(B):** Garbled transfer formula written as min(jug1, jug2_capacity - jug2) and min(jug2, jug1_capacity - jug1).

• **3(A):** Start state 4,5,8 in bottom row changed to 7,5,8. Manhattan used abs(i-y) instead of abs(j-y); `stats` in to_tuple changed to `state`.

• **3(B):** The initial state (1,2,3),(4,0,6),(7,8,5) is an unsolvable 8-puzzle (odd number of inversions), so the program would crash with a TypeError. Changed the last row to (7,5,8), as in 2(A) and 3(A). Also fixed range(2) to range(3) in manhattan and goal_pos(value) to goal_pos[value].

• **3(C):** City names corrected to standard spellings (Zerind, Oradea, Sibiu, Craiova, Fagaras...). Added an Oradea entry to the graph so the search cannot hit a missing key.

• **5:** Kept 'Cold' in the Temperature list exactly as written in the notebook (the standard dataset has 'Cool' there).

• **6(A)/(B):** Alarm CPD value 0.04 corrected to 0.94, John CPD 0.03 to 0.05, Mary CPD 0.20 to 0.70, so each column sums to 1. The notebook values differ between 6(A) and 6(B); the 6(B) values were used.